

Gini Site Glossary

Private calculation glossary for the current Streamlit site. Organized by page, with a shared data layer for formulas that feed multiple pages.

Source Coverage

This glossary covers every current site page routed through app.py, plus the prepared data builder that creates the model fields displayed across those pages.

Site pages	About Me, Gini Dashboard, Live Leaderboard, Super Square, Predictive Model, Model History, Terminology.
Shared source	build_nfl_estat_data.py, team_game_estat.csv, team_season_estat.csv, games, roster, and live source CSVs.
Reading rule	Formula entries are written as equations or plain math, not code. UI-only formatting is excluded unless it changes a displayed calculated value.

Shared Prepared Data Layer

These calculations are created before the pages render and are reused by the Gini Dashboard, Predictive Model, Super Square, and Live Leaderboard.

Play Filtering

[Data prep](#)

Used in: Prepared play-by-play layer

The data builder keeps team-context scrimmage rows, removes quarterback kneels and spikes, and coerces play-value columns to numeric values before aggregation.

Formula / calculation:

```
usable_play = team_context_play AND not_qb_kneel AND not_qb_spike
```

Note: This is a data-quality step, but it affects every downstream calculation.

Offensive EPA per Play

[Game stat](#)

Used in: team_game_estat.csv, weekly chart, model inputs

Average expected points added by the offense in a game.

Formula / calculation:

```
off_epa_total = sum(epa)
off_epa_per_play = mean(epa)
off_plays = count(epa rows)
```

Offensive Success Rate

Game stat

Used in: Game EStat, Success Margin, tables

The average of the play-by-play success flag for the team's offensive plays.

Formula / calculation:

```
off_success_rate = mean(success)
```

Defensive EPA Allowed per Play

Game stat

Used in: Game log, opponent-adjusted defense

Average EPA allowed to the opposing offense when the selected team is on defense.

Formula / calculation:

```
def_epa_allowed_total = sum(opponent epa)
def_epa_allowed_per_play = mean(opponent epa)
def_plays = count(defensive rows)
```

Note: Lower raw values are better, but adjusted defensive fields are oriented so higher is better.

Defensive Success Rate Allowed

Game stat

Used in: Success Margin, game log

How often the opponent's offense was successful against the selected team's defense.

Formula / calculation:

```
def_success_allowed = mean(opponent success)
```

Opponent-Adjusted Offensive EPA

Opponent context

Used in: Offense EStat, Game EStat, Predictive Model

Offensive efficiency adjusted by the defensive quality of the opponent.

Formula / calculation:

```
off_adj_epa = off_epa_per_play - opponent_season_def_epa_allowed_per_play
```

Note: Good offensive production against a defense that usually allows little EPA receives more credit.

Opponent-Adjusted Defensive EPA

Opponent context

Used in: Defense EStat, Game EStat, Super Square

Defensive efficiency adjusted by the offensive quality of the opponent.

Formula / calculation:

```
def_adj_epa = opponent_season_off_epa_per_play - def_epa_allowed_per_play
```

Note: Higher means better defense because the formula rewards holding opponents below their normal offense.

Point Differential

[Scoreboard](#)

Used in: Game log, rankings, predictive features

The scoring margin for one game.

Formula / calculation:

```
point_diff = points_for - points_against
```

Turnover Margin

[Possession value](#)

Used in: Game EStat, Gini Dashboard, Predictive Model

The possession swing from takeaways and offensive turnovers.

Formula / calculation:

```
turnovers = interceptions + fumbles_lost  
turnover_margin = takeaways - turnovers
```

Penalty Yard Margin

[Hidden yardage](#)

Used in: Game EStat, game log, rankings table

Penalty yardage advantage from the opponent's penalties minus the selected team's penalties.

Formula / calculation:

```
penalty_yards_margin = opponent_penalty_yards_committed - team_penalty_yards_committed
```

Note: Positive is favorable because the opponent gave away more penalty yards.

Pass Rate

[Play mix](#)

Used in: Prepared team-game and season data

How often a team passed among pass and rush attempts.

Formula / calculation:

```
pass_rate = pass_attempts / (pass_attempts + rush_attempts)
```

Success Margin

[Consistency](#)

Used in: Game EStat, Gini Dashboard, Super Square, Predictive Model

The team's offensive consistency minus the consistency it allowed on defense.

Formula / calculation:

```
success_margin = off_success_rate - def_success_allowed
```

Raw Game EStat

Game score

Used in: Gini Dashboard weekly chart and game log

A raw game-level performance score that blends adjusted efficiency, consistency, turnovers, and penalty-yard advantage.

Formula / calculation:

```
estat_game = 0.40*off_adj_epa
+ 0.40*def_adj_epa
+ 0.10*success_margin
+ 0.07*turnover_margin
+ 0.03*(penalty_yards_margin / 50)
```

Note: This raw value is later converted to a display scale on the Gini Dashboard.

Season Aggregates

Season data

Used in: team_season_estat.csv

Team-season totals and averages are calculated from the game-level file.

Formula / calculation:

```
games = count(unique game_id)
points_for = sum(points_for)
points_against = sum(points_against)
point_diff = sum(point_diff)
off_adj_epa = mean(off_adj_epa)
def_adj_epa = mean(def_adj_epa)
```

Note: The season file also carries summed yards, penalties, turnovers, and averaged rates.

Per-Game Margins

Season data

Used in: rankings tables and model inputs

Season totals converted to game-level pace fields.

Formula / calculation:

```
point_diff_per_game = point_diff / games
turnover_margin_per_game = turnover_margin / games
penalty_yards_margin_per_game = penalty_yards_margin / games
```

Yards per Play

Season data

Used in: Terminology and predictive features

Basic offensive yardage efficiency.

Formula / calculation:

```
yards_per_play = yards_gained / off_plays
```

Season Z-Score

Standardization

Used in: All 100-scale EStat scores

A season-relative standardization that compares each team to its own season's league environment.

Formula / calculation:

$$z = (\text{team_value} - \text{season_average}) / \text{season_standard_deviation}$$

Note: The builder uses z-scores for offense, defense, point differential, success, turnovers, penalties, and schedule strength.

Offense EStat

Component score

Used in: Gini Dashboard, Super Square, Predictive Model

A 100-scale offensive score based on opponent-adjusted offensive EPA.

Formula / calculation:

```
off_z = zscore_by_season(off_adj_epa)
offense_estat = 100 + 15*off_z
```

Defense EStat

Component score

Used in: Gini Dashboard, Super Square, Predictive Model

A 100-scale defensive score based on opponent-adjusted defensive EPA.

Formula / calculation:

```
def_z = zscore_by_season(def_adj_epa)
defense_estat = 100 + 15*def_z
```

Default EStat

Stored score

Used in: team_season_estat.csv, rankings table

The stored baseline team-season score saved in the prepared CSV.

Formula / calculation:

```
overall_estat = 100 + 15*(
0.30*off_z + 0.30*def_z + 0.15*pd_z
+ 0.15*success_z + 0.07*turnover_z + 0.03*penalty_z
)
```

Note: The stored Default EStat does not include schedule strength. The interactive dashboard Custom EStat can include schedule strength.

Schedule Strength

Schedule context

Used in: Gini Dashboard, rankings, live and predictive pages

The average Default EStat of a team's opponents.

Formula / calculation:

```
schedule_strength = mean(opponent_overall_estat for games on schedule)
schedule_strength_z = zscore_by_season(schedule_strength)
```

Note: Higher means a harder schedule, not a better team by itself.

Season Ranks

[Ranking](#)

Used in: rankings tables, cards, predictors

Ranks are computed within each season with rank 1 as best for the selected metric.

Formula / calculation:

```
overall_rank = rank_desc(overall_estat)
offense_rank = rank_desc(offense_estat)
defense_rank = rank_desc(defense_estat)
sos_rank = rank_desc(schedule_strength)
```

Note: The code uses the minimum rank method for ties.

About Me Page

The About Me page is primarily biographical and explanatory. It does not run football model calculations, but it does describe the model concept.

Core Idea Reference

Static explanation

Used in: About Me hero/body copy

The page explains that Gini Metric combines offense, defense, point differential, success margin, turnovers, penalties, and schedule strength into one evaluation idea.

Formula / calculation:

No numeric football calculation is executed on this page.

Note: The calculation itself appears in the shared data layer and Gini Dashboard sections.

Quick Facts and Questions

Static display

Used in: About Me cards and question grid

Focus, model, tools, background, and site questions are displayed as fixed text.

Formula / calculation:

No numeric football calculation is executed on this page.

Gini Dashboard Page

The Gini Dashboard is the main interactive calculation page. It recalculates Custom EStat, rank order, team context, schedule difficulty, weekly display scores, and matchup adjustment from the prepared data.

Weight Normalization

Interactive model

Used in: Customize Model Weights panel

Slider weights are converted into proportional weights before the Custom EStat blend is calculated.

Formula / calculation:

```
normalized_weight_i = slider_weight_i / sum(all slider weights)
if sum(all slider weights) = 0, all normalized weights = 0
```

Baseline Dashboard Weights

Interactive model

Used in: Customize Model Weights panel

The default philosophy used when the page first loads or weights are reset.

Formula / calculation:

```
Offense = 0.30
Defense = 0.30
Point Diff = 0.15
Success Margin = 0.12
Turnovers = 0.06
Penalties = 0.02
Schedule Strength = 0.05
```

Custom EStat / Gini Score

Interactive model

Used in: Top card, rankings chart, rankings table

The live season score recalculated from the current slider weights.

Formula / calculation:

```
custom_estat = 100 + 15*(
w_off*off_z + w_def*def_z + w_pd*pd_z
+ w_success*success_z + w_to*turnover_z
+ w_penalty*penalty_z + w_schedule*schedule_z
)
```

Note: The dashboard labels this selected-team value as the Gini Score.

Custom Rank

Ranking

Used in: Overall rankings and selected-team card

The league rank after applying the current Custom EStat calculation.

Formula / calculation:

```
custom_rank = rank_desc(custom_estat) within selected season
```

Note: Tie handling uses the minimum rank method.

Selected-Team Record

Top card

Used in: Gini Dashboard hero

The selected team's win-loss-tie record is calculated from game scores.

Formula / calculation:

```
wins = count(points_for > points_against)
losses = count(points_for < points_against)
ties = count(points_for = points_against)
record = W-L or W-L-T
```

Gini Average Delta Badge

Context badge

Used in: Selected-team Gini Score card

Compares the selected team's Custom EStat to the average Custom EStat for that selected season.

Formula / calculation:

```
custom_average = mean(custom_estat for selected season)
gini_average_delta = selected_team_custom_estat - custom_average
```

Note: If the absolute delta is under 0.05, the badge reads as at the season Gini average.

Rank Badge Buckets

[Card display](#)

Used in: Offense, defense, and deep-dive cards

The rank badge color/text bucket is based on where the team ranks in the league.

Formula / calculation:

```
rank <= 12: positive bucket
13 <= rank <= 21: neutral bucket
rank >= 22: negative bucket
```

Schedule Z-Score

[Interactive model](#)

Used in: Custom EStat schedule component

Schedule strength is standardized inside the selected season before it can be blended into Custom EStat.

Formula / calculation:

```
schedule_z = (schedule_strength - mean(schedule_strength)) / std(schedule_strength)
if std is missing or 0, schedule_z = 0
```

Schedule Difficulty Label

[Schedule context](#)

Used in: Top card

Turns schedule strength rank into an easy-to-read schedule label.

Formula / calculation:

```
rank 1: Hardest Schedule
rank <= 12: Hard Schedule
rank <= 21: Average Schedule
rank 32: Easiest Schedule
otherwise: Easy Schedule
```

Note: This is a context label, not a team-quality grade.

Team Balance

[Deep dive](#)

Used in: Team Deep Dive top cards

A label that compares offense and defense rank to describe the selected team's profile.

Formula / calculation:

```
balance_gap = abs(offense_rank - defense_rank)
if offense_rank <= 12 and defense_rank <= 12: Two-Way
elif balance_gap <= 5: Balanced
elif offense_rank < defense_rank: Offense-Led
else: Defense-Led
```

Top-N Rankings Chart

[Display calculation](#)

Used in: Overall Rankings tab

The chart automatically includes at least the selected team rank and at least 15 teams unless the user changes the slider.

Formula / calculation:

```
default_top_n = max(15, selected_team_custom_rank)
chart_rows = top_n teams by custom_rank
```

Offense vs Defense Logo Size

[Chart display](#)

Used in: Offense vs Defense tab

Logo size is based on positive point differential per game, with losing-margin teams held at a minimum size.

Formula / calculation:

```
bubble_size = max(point_diff_per_game, 0) + 1
Logo Scale = 1.00 + 1.25*((bubble_size - min_bubble)/(max_bubble - min_bubble))
```

Note: If all bubble sizes are equal, Logo Scale defaults to 1.25.

Opponent Strength

[Weekly adjustment](#)

Used in: Team Deep Dive matchup checkbox

Each game's opponent receives the opponent's current Custom EStat from the selected season.

Formula / calculation:

```
opponent_strength = custom_estat(opponent)
if missing: opponent_strength = league_average_custom_estat
```

Matchup Difficulty Adjustment

[Weekly adjustment](#)

Used in: Team Deep Dive matchup checkbox

A small display-scale bonus or penalty based on whether the opponent is stronger or weaker than league average.

Formula / calculation:

```
opponent_strength_adj = opponent_strength - league_average_custom_estat
matchup_difficulty_adjustment = 0.20 * opponent_strength_adj
```

Note: This current page applies the adjustment after converting raw Game EStat onto the display scale.

Game EStat Display Scale

[Weekly display](#)

Used in: Weekly chart and game log

Converts raw Game EStat to an easier-to-read weekly number.

Formula / calculation:

```
estat_game_display = 70 + 60*estat_game
opp_adj_estat_game_display = estat_game_display + matchup_difficulty_adjustment
```

Note: A raw Game EStat of 0.0 displays as 70.0; a raw value of 0.5 displays as 100.0.

Deep-Dive Metric Ranks

[Deep dive](#)

Used in: Point Diff/G, Adj Off EPA, Adj Def EPA, TO Margin/G cards

The selected team's rank is recalculated for whichever metric the card displays.

Formula / calculation:

```
metric_rank = rank_desc(metric) if higher_is_better
metric_rank = rank_asc(metric) if lower_is_better
```

Note: The dashboard uses higher-is-better for the deep-dive metrics shown.

Game Log Week Filter

[Table filter](#)

Used in: Team Deep Dive game log

The selected week range filters the game-log table.

Formula / calculation:

```
game_log_rows = games where min_selected_week <= week <= max_selected_week
```

Schedule Table Display Cleanup

[Display cleanup](#)

Used in: Schedule + Roster tab

Scores are converted to whole-number text, game time is shortened to HH:MM, roof text is capitalized, and surface is simplified.

Formula / calculation:

```
away_score/home_score -> integer text
gametime -> first 5 characters
roof -> capitalized
surface -> Grass, Turf, or capitalized source value
```

Note: These are display calculations only.

Roster Table Display Cleanup

[Display cleanup](#)

Used in: Schedule + Roster tab

Roster filters and display labels are cleaned for reading.

Formula / calculation:

```
filter_position = selected position unless All
filter_name = case-insensitive player search
status_code -> status_label
missing draft team -> Undrafted
```

Live Leaderboard Page

The Live Leaderboard builds a current-market score from Gini performance, roster strength, projected wins, Super Square quadrant probability, and movement from snapshots.

Live Gini Score Fallback

Performance layer

Used in: Live Leaderboard

The page uses an available Gini-style score in priority order, then reconstructs one if needed.

Formula / calculation:

```
if custom_estat exists: current_gini_score = custom_estat
elif overall_estat exists: current_gini_score = overall_estat
elif estat_raw exists: current_gini_score = estat_raw
else: reconstruct from weighted season z-scores
```

Reconstructed Gini-Style Score

Performance layer

Used in: Live Leaderboard fallback

Fallback score if the normal Gini fields are missing.

Formula / calculation:

```
score = 0.22*z(offense_estat) + 0.22*z(defense_estat)
+ 0.14*z(point_diff_per_game) + 0.12*z(success_margin)
+ 0.08*z(turnover_margin_per_game)
- 0.05*z(penalty_yards_margin_per_game)
+ 0.07*z(schedule_strength)
+ 0.05*z(off_adj_epa) + 0.05*z(def_adj_epa)
current_gini_score = 100 + 15*(score / sum_used_weights)
```

Note: The penalty-yard component is inverted in this fallback.

Performance Score

Performance layer

Used in: Live Market Score

Normalizes current Gini score to a live-market scale.

Formula / calculation:

```
performance_score = normalize_score(current_gini_score, center=85, spread=10, lower=60, upper=120)
```

Normalize Score

Scaling

Used in: Performance, production, and other score layers

Generic conversion of a distribution into a clipped score scale.

Formula / calculation:

```
normalized = center + spread*((value - mean(values)) / std(values))
normalized = clip(normalized, lower, upper)
if std is missing or 0: normalized = center
```

Current Record

Record layer

Used in: Live cards and team detail

Current wins, losses, ties, and scored games are calculated from team_game scores.

Formula / calculation:

```
win = 1 if points_for > points_against else 0
loss = 1 if points_for < points_against else 0
tie = 1 if points_for = points_against else 0
current_wins/losses/ties = sum by season-team
```

Player Production Raw Score

Roster layer

Used in: Roster Score when player stats are available

Player production starts as a weighted sum across passing, rushing, receiving, defense, fantasy, and EPA fields when those fields exist.

Formula / calculation:

```
production_raw = sum(stat_column * stat_weight)
examples: passing_yards*0.006, passing_tds*1.6,
interceptions*(-2.4), sacks*2.0, rushing_yards*0.010,
receiving_yards*0.010, fantasy_points*0.18, epa*10.0
```

Note: Only columns present in the source data are used.

Player Production Score

Roster layer

Used in: Roster Score

Production raw score is normalized after summing by player.

Formula / calculation:

```
production_score = normalize_score(production_raw, center=75, spread=12, lower=45, upper=100)
```

Availability Multiplier

Roster layer

Used in: Player score and team roster score

Player availability changes how much that player's score counts.

Formula / calculation:

```
Active/Full = 1.00
Questionable = 0.82
Doubtful = 0.35
IR/Reserve/PUP/Suspended = 0.14
Practice squad = 0.35
Free agent/cut/waived/out = 0.08
Fallback unknown status = 0.88
```

Note: The source code includes additional status aliases such as RFA, UDF, and RSN.

Experience Score

[Roster layer](#)

Used in: Player score fallback

A piecewise player experience score based on years of experience.

Formula / calculation:

```
if exp <= 1: score = 58 + 5*exp
elif exp <= 4: score = 65 + 6*(exp - 1)
elif exp <= 8: score = 83 + 1.5*(8 - abs(6 - exp))
else: score = 86 - 2.2*(exp - 8)
experience_score = clip(score, 52, 90)
```

Draft Capital Score

[Roster layer](#)

Used in: Player score fallback

A piecewise score based on draft pick number. Undrafted or missing pick defaults lower.

Formula / calculation:

```
missing pick -> 48
pick 1-32: 95 to 85
pick 33-64: 85 to 75
pick 65-100: 78 to 68
pick 101-170: 68 to 58
pick 171-260: 55 to 45
pick > 260: 45
```

Note: Each range declines linearly across the pick interval.

Position Importance

[Roster layer](#)

Used in: Rookie and player score weighting

Premium positions receive a multiplier.

Formula / calculation:

```
QB = 1.14
EDGE/DE/OT/T/CB = 1.08
WR/DL/DT/DB = 1.04
OL/front/secondary groups = 1.02
special teams = 0.78
skill = 0.98
other = 0.92
```

Rookie Projection Score

[Roster layer](#)

Used in: Roster Score

A projection score for rookies and very new players.

Formula / calculation:

```
rookie_projection_score = clip(draft_capital_score * position_importance, 40, 98)
```

Player Score Blend

Roster layer

Used in: Position group scores

Blends the fallback player score with production when production data exists.

Formula / calculation:

```
fallback = 0.45*draft_capital_score + 0.40*experience_score + 0.15*rookie_projection_score
if no production: player_score = clip(fallback, 35, 98)
else: player_score = clip(fallback_weight*fallback + (1-fallback_weight)*production_score, 35, 100)
```

Note: fallback_weight is 0.90 through 2 games, 0.65 through 5, 0.40 through 9, and 0.15 after that.

Weighted Top Average

Roster layer

Used in: Position group scores

Blends top-end talent and group depth for each position group.

Formula / calculation:

```
top_average = weighted average of top_n scores, weights from 1.00 down to 0.42
group_depth_average = mean(top_n scores)
weighted_top_average = clip(0.72*top_average + 0.28*group_depth_average, 45, 100)
```

Note: If a group has no players, it returns a neutral default.

Continuity Score

Roster layer

Used in: Roster Score

Measures how much of the previous year's roster carried over.

Formula / calculation:

```
overlap_ratio = returning_players / previous_roster_players
continuity_score = 50 + 50*clip(overlap_ratio, 0, 1)
```

Note: Falls back to name matching if player IDs are unavailable.

Position Group Scores

Roster layer

Used in: Team detail and Roster Score

Team roster strength by position group.

Formula / calculation:

```
weighted_player_score = player_score * availability_multiplier
qb_score = weighted_top_average(QB, top 2, neutral 68)
skill = top 8, OL = top 8, defensive front = top 9,
secondary = top 8, special teams = top 3,
premium positions = top 10
```

Team Roster Score

Roster layer

Used in: Live Market Score and team detail

The final team roster score blends key position groups, availability, rookies, and continuity.

Formula / calculation:

```
base = 0.18*qb_score + 0.17*offensive_line_score
+ 0.13*offense_skill_score + 0.17*defensive_front_score
+ 0.14*secondary_score + 0.08*premium_position_score
+ 0.08*availability_score + 0.03*rookie_score
+ 0.02*continuity_score
roster_score = clip(base + transaction_impact, 50, 100)
```

Note: transaction_impact is currently 0.0 in the page.

Projected Wins Score

Projection layer

Used in: Live Market Score

Projected wins are converted to a 50-100 score.

Formula / calculation:

```
projected_wins_score = clip(50 + (projected_wins / 17)*50, 50, 100)
```

Live Projected Wins Model

Projection layer

Used in: Live Leaderboard

The live page trains Ridge, Gradient Boosting, and Random Forest models on historical next-season wins, chooses the lowest validation MAE, and predicts current teams.

Formula / calculation:

```
next_season_wins = following season current_wins
model_prediction = best_model(features)
if best_model_mae > previous_wins_baseline_mae:
    projected_wins = 0.65*model_prediction + 0.35*current_wins_baseline
else:
    projected_wins = model_prediction
projected_wins = clip(projected_wins, 0, 17)
```

Note: If modeling is unavailable, projected wins fall back to current wins or 8.5.

Projected Wins Range

Projection layer

Used in: Live Leaderboard

The projected win interval uses the model's MAE, clipped to a reasonable range.

Formula / calculation:

```
mae = clip(model_mae, 1.4, 3.5)
projected_wins_low = round(projected_wins - mae), clipped 0-17
projected_wins_high = round(projected_wins + mae), clipped 0-17
```

Live Super Square Inputs

Quadrant layer

Used in: Live quadrant probabilities

The live page builds Super Square-style inputs from regular-season game data.

Formula / calculation:

```
epa_diff_game = off_adj_epa + def_adj_epa
best_epa_unit_game = max(off_adj_epa, def_adj_epa)
pd_pos_share = mean(point_diff > 0)
Gini Score = 100 + 15*(0.35*off_z + 0.35*def_z + 0.15*pd_z + 0.15*success_z)
```

Note: These are similar to the Super Square page's calculations.

Champion-Derived Quadrant Cutoffs

Quadrant layer

Used in: Live quadrant probabilities

Requirements are learned from historical champion rows. Low-rank requirements use the worst champion rank as the cutoff; high-value requirements use the lowest champion value as the cutoff.

Formula / calculation:

```
if direction = low: cutoff = max(champion values)
if direction = high: cutoff = min(champion values)
```

Control and Unit Pressure Scores

Quadrant layer

Used in: Live quadrant probabilities

Requirement scores are converted into two axis scores.

Formula / calculation:

```
low-rank requirement score = 100 + 5*(cutoff - value)
high-value requirement score = 100 + 18*((value - cutoff) / scale)
Control Profile Score = min(control requirement scores)
Unit Pressure Score = min(pressure requirement scores)
```

Note: Scores are clipped from 30 to 170 in the live logic.

Quadrant Probabilities

Quadrant layer

Used in: Live Leaderboard team detail

Control and pressure scores become probabilities for four current profile quadrants.

Formula / calculation:

```
pc = sigmoid((Control Profile Score - 100) / 8)
pp = sigmoid((Unit Pressure Score - 100) / 8)
Q1 = (1-pc)*(1-pp)
Q2 = pc*(1-pp)
Q3 = (1-pc)*pp
Q4 = pc*pp
probabilities are normalized to sum to 1
```

Quadrant Probability Score

Quadrant layer

Used in: Live Market Score

The four quadrant probabilities are converted into a single 50-100 contender-profile score.

Formula / calculation:

```
quadrant_probability_score = clip(
60*Q1 + 75*Q2 + 88*Q3 + 100*Q4,
50, 100
)
```

Live Market Score

Final live score

Used in: Live Leaderboard rank order

The main live ranking score.

Formula / calculation:

```
live_market_score = clip(
0.65*performance_score + 0.10*roster_score
+ 0.15*projected_wins_score + 0.10*quadrant_probability_score,
45, 120
)
```

Live Rank

Ranking

Used in: Live Leaderboard

Teams are ranked by Live Market Score.

Formula / calculation:

```
live_rank = ordinal rank after sorting live_market_score descending
```

Rank and Score Movement

Snapshot layer

Used in: Biggest movers and row movement

Movement compares current rank/score with the previous snapshot for the same season.

Formula / calculation:

```
rank_change = previous_rank - live_rank
score_change = live_market_score - previous_score
arrow = up if rank_change > 0, down if rank_change < 0, otherwise right
```

Status Label

Final label

Used in: Live row and team detail

A rule-based label summarizes the team profile.

Formula / calculation:

```
Elite Market if live_market_score >= 96
Q4 Threat if Q4 is most likely and probability >= 0.34
High Upside if projected_wins - current_wins >= 2 and roster_score >= 78
Rising if movement arrow is up
Regression Watch if projected_wins - current_wins <= -2
Roster Boost if roster_score >= performance_score + 8
Watch List if live_market_score < 72
otherwise Stable
```

Super Square Page

The Super Square page turns regular-season profiles into a two-axis contender map using historical Super Bowl champion cutoffs.

Super Square Season Window

Data scope

Used in: Super Square

The page builds profiles for seasons 2005 through 2025 from regular-season team-game rows.

Formula / calculation:

```
included_season = 2005 <= season <= 2025
included_game = season_type = REG
```

Checkpoint Metrics

Checkpoint profile

Used in: Super Square requirements

For selected weeks, the page averages team performance up to that week. It also averages late-season performance from a starting week onward.

Formula / calculation:

```
through-week checkpoints for weeks 5, 6, 8, 10, 12, 14:
checkpoint_estat = mean(estat_game through week)
checkpoint_epa = mean(off_adj_epa + def_adj_epa through week)
checkpoint_unit = mean(max(off_adj_epa, def_adj_epa) through week)
checkpoint_point_diff = mean(point_diff through week)
checkpoint_success = mean(success_margin through week)
late checkpoints repeat from weeks 10, 12, 14 onward
```

Note: Each checkpoint value also receives a season rank.

Positive Point-Differential Share

Scoreboard control

Used in: Super Square control axis

The share of games where the team had a positive point differential.

Formula / calculation:

```
pd_pos_share = mean(point_diff > 0)
pd_pos_share_rank = rank_desc(pd_pos_share)
```

Super Square EPA Differential

Efficiency

Used in: Super Square requirements and hover

Combined adjusted offensive and defensive EPA.

Formula / calculation:

```
epa_diff_game = off_adj_epa + def_adj_epa
EPA Differential = mean(off_adj_epa) + mean(def_adj_epa)
```

Best EPA Unit

Unit strength

Used in: Super Square pressure axis

The better of offense and defense by adjusted EPA or by 100-scale EStat.

Formula / calculation:

```
best_epa_unit_game = max(off_adj_epa, def_adj_epa)
best_epa_unit = max(mean(off_adj_epa), mean(def_adj_epa))
Best Unit Score = max(Offense EStat, Defense EStat)
```

Super Square Gini Score

Composite score

Used in: Super Square control profile

A Super Square-specific 100-scale Gini score based on offense, defense, point differential, and success.

Formula / calculation:

```
off_z = z(off_epa)
def_z = z(def_epa)
pd_z = z(point_diff_per_game)
success_z = z(success_margin)
Gini Score = 100 + 15*(0.35*off_z + 0.35*def_z + 0.15*pd_z + 0.15*success_z)
```

Note: This Super Square score is built from game-derived fields inside the Super Square page, not the slider Custom EStat.

Champion-Derived Requirements

Research step

Used in: Super Square gates

The page looks at historical champion rows and sets each requirement's cutoff once.

Formula / calculation:

```
low-rank requirement cutoff = max(champion rank/value)
high-value requirement cutoff = min(champion value)
```

Note: The current requirements include scoreboard consistency, Week 12 EStat rank, late-season EPA floor, success margin rank, Week 8 EStat floor, Week 14 EPA rank, Week 10 unit strength, and late unit strength.

Requirement Pass

Gate rule

Used in: Super Square gates

Determines whether a team clears each individual championship-profile requirement.

Formula / calculation:

```
if direction = low: pass = value <= cutoff
if direction = high: pass = value >= cutoff
```

Requirement Miss

Gate rule

Used in: Super Square gates

Measures how far a team misses a requirement.

Formula / calculation:

```
if direction = low: miss = max(value - cutoff, 0)
if direction = high: miss = max((cutoff - value) / max(abs(cutoff), 0.05), 0)
```

Gate Score

Axis score

Used in: Control Profile and Unit Pressure axes

Each requirement becomes a 30-170 score centered around 100.

Formula / calculation:

```
if direction = low:
gate_score = 100 + 5*(cutoff - value)
if direction = high:
gate_score = 100 + 18*((value - cutoff) / scale)
where scale = max(season_std, abs(cutoff), 0.05)
gate_score = clip(gate_score, 30, 170)
```

Gate Pass Count and Miss Score

Summary

Used in: Internal Super Square calculations

Counts how many gates a team passed and totals how much it missed.

Formula / calculation:

```
gate_pass_count = sum(gate_pass booleans)
gate_miss_score = sum(gate_miss values)
```

Control Profile Score

Chart axis

Used in: Horizontal axis

The weakest control-axis requirement score becomes the team's horizontal score.

Formula / calculation:

```
Control Profile Score = min(gate scores where axis = control)
```

Note: Using the minimum makes the axis a threshold profile: one weak control gate can hold the score down.

Unit Pressure Score

Chart axis

Used in: Vertical axis

The weakest pressure-axis requirement score becomes the team's vertical score.

Formula / calculation:

```
Unit Pressure Score = min(gate scores where axis = pressure)
```

Clears Axis

Quadrant rule

Used in: Super Square quadrants

A team clears an axis when its composite axis score reaches the fixed 100 line.

Formula / calculation:

```
Clears Control Axis = Control Profile Score >= 100  
Clears Unit Pressure Axis = Unit Pressure Score >= 100
```

Note: The quadrant lines are fixed at 100 and do not move by season.

Inside Super Square

Contender flag

Used in: Summary cards and chart styling

A team is inside only if it passes every underlying gate.

Formula / calculation:

```
Inside Super Square = all(gate_1 through gate_8 are True)
```

Quadrant

Contender map

Used in: Super Square chart

The team quadrant is based on which fixed axes it clears.

Formula / calculation:

```
Q4 = clears control and clears unit pressure  
Q3 = does not clear control but clears unit pressure  
Q2 = clears control but does not clear unit pressure  
Q1 = clears neither
```

On the Cusp

Contender flag

Used in: Summary cards and chart styling

The current page marks Q3 teams as on the cusp.

Formula / calculation:

```
On the Cusp = Quadrant == 'Q3 - Playoff Contenders'
```

Champion Validation

Research proof

Used in: Super Square hero claim

The page validates whether each known champion row was inside the Super Square.

Formula / calculation:

```
champion_inside = Inside Super Square for champion team-season  
all_valid_champions_inside = all(champion_inside for valid champion rows)
```

Note: The page also counts teams inside and teams on the cusp for the selected season.

Predictive Model Page

The Predictive Model page builds historical team-season features, trains rolling backtest models, predicts following-season regular-season wins, and predicts playoff wins for completed seasons.

Regular-Season Record

Target data

Used in: Predictive feature table

Wins, losses, ties, scored games, points for, and points against are calculated from regular-season game rows.

Formula / calculation:

```
current_wins = sum(points_for > points_against)
current_losses = sum(points_for < points_against)
current_ties = sum(points_for = points_against)
scored_games = count(unique game_id)
```

Postseason Wins and Games

Playoff target

Used in: Playoffs Predictor Tool

The page counts playoff appearances and playoff wins from non-regular-season games.

Formula / calculation:

```
postseason_games = count(playoff games appeared in)
postseason_wins = count(playoff games won)
```

Pythagorean Win Percentage

Expected wins

Used in: Regular Season Predictor Tool

Expected win percentage from points scored and allowed.

Formula / calculation:

```
exponent = 2.37
pythagorean_win_pct = points_for^exponent / (points_for^exponent + points_against^exponent)
```

Pythagorean Wins

Expected wins

Used in: Regular Season Predictor Tool

Expected wins scaled by games played.

Formula / calculation:

```
pythagorean_wins = pythagorean_win_pct * games_played_for_model
```

Wins Above Expected

Expected wins

Used in: Regular Season Predictor Tool

How far the team's actual wins were above or below its pythagorean expected wins.

Formula / calculation:

```
win_over_pythagorean = current_wins - pythagorean_wins
```

EPA Differential

Efficiency feature

Used in: Regular and playoff models

Combined adjusted offensive and defensive EPA.

Formula / calculation:

```
epa_differential = off_adj_epa + def_adj_epa
```

Scoring Margin Signal

Composite feature

Used in: Regular-season model features

A compact feature combining scoreboard margin and success-rate margin.

Formula / calculation:

```
scoring_margin_signal = point_diff_per_game + 25*success_margin
```

Gini Score and Gini Rank

Model feature

Used in: Regular and playoff models

The predictor chooses the available Gini-style score and ranks it within each season.

Formula / calculation:

```
if custom_estat exists: gini_score = custom_estat
elif overall_estat exists: gini_score = overall_estat
elif estat_raw exists: gini_score = estat_raw
gini_rank = rank_desc(gini_score) within season
```

Point Differential Rank

Model feature

Used in: Regular and playoff models

Season rank by point differential per game.

Formula / calculation:

```
point_diff_rank = rank_desc(point_diff_per_game) within season
```

Balance and Unit Flags

Binary features

Used in: Regular and playoff models

Offense/defense rank features converted into flags.

Formula / calculation:

```
balance_gap = abs(offense_rank - defense_rank)
elite_offense = offense_rank <= 8
elite_defense = defense_rank <= 8
two_way_team = offense_rank <= 12 AND defense_rank <= 12
strong_unit = offense_rank <= 12 OR defense_rank <= 12
```

Super Square Profile Flag

Binary feature

Used in: Regular and playoff models

A simple historical contender flag inside the predictive model.

Formula / calculation:

```
top_gini_team = gini_rank <= 6
top_point_diff_team = point_diff_rank <= 6
super_square_profile = top_gini_team AND top_point_diff_team AND strong_unit
```

Profile Win Gap Signals

Binary features

Used in: Regular-season model and risk label

Compares a team's win rank to its Gini rank.

Formula / calculation:

```
wins_rank = rank_desc(current_wins) within season
profile_win_gap = wins_rank - gini_rank
underperformer_signal = profile_win_gap >= 4
overperformer_signal = profile_win_gap <= -4
```

Note: A positive gap means the team's profile ranked better than its record rank.

Following-Season Targets

Training target

Used in: Regular-season prediction

The model learns from each team's next season wins only when the next row is exactly the following season.

Formula / calculation:

```
next_season = next team-season year
next_season_wins = next row current_wins
if next_season != season + 1: next_season_wins = missing
wins_change = next_season_wins - current_wins
strong_next_season = next_season_wins >= 10
```

Regular-Season Feature Set

Model input

Used in: Regular Season Predictor Tool

The expanded model considers scoreboard, expected wins, Gini, EPA, success, schedule, turnover, penalty, rank, balance, Super Square, and profile-gap features.

Formula / calculation:

```
features include current_wins, pythagorean_wins, win_over_pythagorean, point_diff_per_game, points_for,
points_against, gini_score, gini_rank, offense_estat, defense_estat, off_adj_epa, def_adj_epa,
epa_differential, success_margin, schedule_strength, ranks, balance flags, super_square_profile, and
profile-gap signals
```

Note: The page only uses features present in the built dataset.

Rolling Regular-Season Backtest

Model validation

Used in: Model check expander

Candidate models are backtested season by season, training only on earlier seasons.

Formula / calculation:

```
for each test_season after at least 8 training seasons:
train on seasons < test_season
predict next_season_wins for test_season
errors = actual_next_season_wins - predicted_next_wins
MAE = mean(abs(errors))
RMSE = sqrt(mean(errors^2))
Within 2 Wins = mean(abs(errors) <= 2)
Correlation = corr(actual, predicted)
```

Note: The best model is selected by lowest Rolling MAE, then Rolling RMSE.

Regular-Season Model Candidates

Model validation

Used in: Regular Season Predictor Tool

The page tests multiple model families.

Formula / calculation:

```
Pythagorean Ridge
Pythagorean Elastic Net
Expanded Ridge
Expanded Elastic Net
Gradient Boosting Regressor
Random Forest Regressor
```

Note: All pipelines use median imputation; linear models also use standard scaling.

Previous-Wins Baseline MAE

Model validation

Used in: Model check expander

The model is compared to simply using previous wins as the forecast.

Formula / calculation:

```
baseline_prediction = current_wins
baseline_mae = MAE(next_season_wins, baseline_prediction)
mae_edge = baseline_mae - model_mae
```

10+ Win Probability

Classifier

Used in: Regular Season Predictor Tool

A Random Forest classifier estimates whether similar profiles produce at least 10 wins next season.

Formula / calculation:

```
strong_next_season = next_season_wins >= 10
prob_10_plus_wins = classifier_probability(strong_next_season = 1)
```

Selected Regular-Season Projection

User-facing forecast

Used in: Regular Season Predictor Tool

The selected team forecast uses the previous full-season profile to project the selected target year.

Formula / calculation:

```
projected_wins = clip(model.predict(selected_feature_row), 0, 17)
likely_range = round(projected_wins +/- model_mae, clipped 0-17)
```

Current Pace Comparison

User-facing forecast

Used in: Regular Season Predictor Tool

If the selected target season is in progress, the page compares current win pace against projection.

Formula / calculation:

```
expected_games = 17 if season >= 2021 else 16
current_win_pace = (actual_wins / games_played) * expected_games
pace_difference = current_win_pace - projected_wins
```

Risk Label

Interpretation

Used in: Regular Season Predictor Tool

A rule-based label summarizes carry-forward risk or upside.

Formula / calculation:

```
Strong carry-forward profile if high Gini/point-diff profile and balanced
Regression risk if >=10 wins but weaker underlying profile
Improvement candidate if underperformer and high profile
Volatile profile if one-sided with elite unit
otherwise Stable profile
```

Note: The text explanation adds reasons such as Super Square profile, two-way team, scoring margin, under/overperformance, and predicted change.

Completed Regular Season Check

Playoff gate

Used in: Playoffs Predictor Tool

The playoff predictor stays locked until the selected regular season is complete.

Formula / calculation:

```
games_played = scored_games, or wins + losses + ties
season_complete = games_played >= expected_regular_season_games(season)
```

Playoff Feature Set

Model input

Used in: Playoffs Predictor Tool

The playoff model uses selected/current season team quality to predict postseason wins.

Formula / calculation:

```
features include current_wins, pythagorean_wins, win_over_pythagorean, point_diff_per_game, gini_score,
ranks, offense/defense Estat, adjusted EPA, EPA differential, success margin, schedule strength, balance
flags, Super Square flags, profile-gap signals, turnovers, and penalties
```

Rolling Playoff Backtest

Model validation

Used in: Playoffs Predictor Tool

Playoff models are backtested by season using completed regular-season profiles.

Formula / calculation:

```
predicted_postseason_wins = model(features), clipped 0-4
errors = postseason_wins - predicted_postseason_wins
Rolling MAE = mean(abs(errors))
Rolling RMSE = sqrt(mean(errors^2))
Playoff Team MAE = MAE only among teams with postseason_games > 0
```

Note: The best model is selected by Rolling MAE, then Playoff Team MAE.

Playoff Model Candidates

Model validation

Used in: Playoffs Predictor Tool

Candidate regression models for playoff wins.

Formula / calculation:

```
Playoff Ridge
Playoff Elastic Net
Playoff Gradient Boosting Regressor
Playoff Random Forest Regressor
```

Playoff Win Probability

Classifier

Used in: Playoffs Predictor Tool

A classifier estimates the chance of at least one postseason win.

Formula / calculation:

```
won_playoff_game = postseason_wins >= 1
playoff_win_probability = classifier_probability(won_playoff_game = 1)
```

Note: The classifier candidates include Logistic Regression, Random Forest, and Gradient Boosting.

Projected Playoff Wins and Range

User-facing forecast

Used in: Playoffs Predictor Tool

The selected team playoff forecast is clipped to the NFL playoff-win range.

Formula / calculation:

```
projected_playoff_wins = clip(model.predict(row), 0, 4)
playoff_range = round(projected_playoff_wins +/- playoff_mae), clipped 0-4
```

Playoff Outlook Label

[Interpretation](#)

Used in: Playoffs Predictor Tool

A simple threshold label based on projected playoff wins and win probability.

Formula / calculation:

```
Deep-run contender if projected_wins >= 1.3 OR probability >= 0.58
Playoff win threat if projected_wins >= 0.8 OR probability >= 0.38
Outside chance if projected_wins >= 0.35 OR probability >= 0.20
otherwise Long-shot profile
```

Model History Page

The Model History page is a narrative documentation page. It explains the evolution of the model but does not execute football calculations.

Model Stages

[Static documentation](#)

Used in: Model History timeline

The page renders prewritten stage descriptions for the Original Gini Stat, Simple Gini Stat, Pass and Run Splits, Advanced Gini Stat, and current Gini Metric.

Formula / calculation:

No numeric football calculation is executed on this page.

Note: The page's purpose is explanation, not computation.

Key Findings

[Static documentation](#)

Used in: Model History cards

The findings cards are fixed explanatory text about record, context, balance, details, and public clarity.

Formula / calculation:

No numeric football calculation is executed on this page.

Terminology Page

The Terminology page is a public-facing dictionary of the metrics. It does not calculate new values, but it names and explains many fields calculated elsewhere.

Current Model Terms

[Static glossary](#)

Used in: Terminology tabs

The page defines Gini Metric, Model Weights, Score Scale, selected-team card, schedule difficulty, team balance, Custom EStat, Default EStat, Offense/Defense EStat, season standardization, Game EStat, and Matchup-Adjusted Game EStat.

Formula / calculation:

No numeric football calculation is executed on this page.

Note: The formulas for those terms appear in the shared layer and Gini Dashboard sections.

Game, Season, and Context Terms

[Static glossary](#)

Used in: Terminology tabs

The page defines EPA, success rate, point differential, turnover margin, penalty yard margin, pass rate, schedule fields, betting market fields, rest/division fields, and roster fields.

Formula / calculation:

No numeric football calculation is executed on this page.

Navigation / App Routing

The app router and shared navigation define page order and active links. They do not calculate football metrics.

Current Page Routes

[Routing](#)

Used in: `app.py` and `site_nav.py`

The site routes are About Me, Gini Dashboard, Live Leaderboard, Super Square, Predictive Model, Model History, and Terminology.

Formula / calculation:

No football calculation is executed by the router or shared navigation.

Note: The active link styling is UI state, not a model calculation.

Old Gini Site / Legacy Gini Stats

Addendum to Gini Site Glossary. This section preserves the old public Gini Stat framework and the 2025-2026 summary stats from the legacy website.

Source

Old Gini Site: <https://congini.github.io/NFL-Gini-Stat/>

Legacy Framework

The old site describes the original NFL EStat/Gini system as an opponent-adjusted efficiency framework that blended traditional production, advanced play value, penalties, turnovers, schedule context, and offense/defense splits.

Legacy Offensive EStat

Old EStat

Used in: Old Gini Site simple equations

The original offensive EStat compared a team's offensive production bundle to a league-average offensive bundle, then added or subtracted football-context factors.

Formula / calculation:

```
Offensive production bundle = Games + Off Yds/G + Off Pts/G
+ Offensive Total Plays + Offensive Total Points
League offensive bundle = league-average version of the same bundle
Penalty Ratio = Team Forward Penalty Yards / Team Allowed Penalty Yards
```

```
EStat Offense = (Offensive production bundle / League offensive bundle)
- 0.10*Turnovers
+ 0.50*TDs_per_Game
+ 0.25*Points_per_Field_Goal
+ Penalty Ratio
+ Offensive EPA_per_Play
```

Note: The old site calls the league-average bundle 'The Number.'

Legacy Defensive EStat

Old EStat

Used in: Old Gini Site simple equations

The original defensive EStat inverted an allowed-production bundle so lower allowed production increased the score, then added takeaways, pressure, sacks, and penalty context.

Formula / calculation:

```
Allowed production bundle = Games + Total Points Allowed + Pts/G Allowed
+ Total Rush Yds Allowed + Rush Yds/G Allowed
+ Total Pass Yds Allowed + Pass Yds/G Allowed
+ Total Yds Allowed + Yds/G Allowed
+ Total Plays Allowed
```

```
EStat Defense = 2*(League defensive bundle / Allowed production bundle)
+ 0.05*Takeaways
+ 0.25*Penalty Ratio
- 0.01*Missed Tackles
+ 0.006*Total Pressures
- EPA_per_Play_Allowed
+ 0.05*Total Sacks
```

Note: This keeps the old direction: better defense should produce a higher defensive EStat.

Legacy 4.5 Average Scaling

Old scale

Used in: Old EStat offense and defense

The old EStats were first rescaled so the league average equaled 4.5.

Formula / calculation:

```

EStat multiplier_4.5 = 4.5 / mean(unadjusted EStat across all 32 teams)
Adjusted EStat Offense = EStat Offense * EStat multiplier_4.5
Adjusted EStat Defense = EStat Defense * EStat multiplier_4.5

```

Legacy Offensive Gini Stat

Old Gini

Used in: Old Gini Site basic equations

The old offensive Gini Stat adjusted the 4.5-scaled offensive EStat by the average defensive EStat of opponents played so far.

Formula / calculation:

```

Gini Stat Offense = Adjusted EStat Offense /
((1 - Average Defensive EStat of Teams Played So Far)*0.1)

```

Note: This formula directly folds schedule/opponent defensive strength into the offensive grade.

Legacy Defensive Gini Stat

Old Gini

Used in: Old Gini Site basic equations

The old defensive Gini Stat adjusted defensive EStat by the average offensive EStat of opponents played so far.

Formula / calculation:

```

Gini Stat Defense = EStat Defense / Average Offensive EStat of Teams Played So Far

```

Legacy 70 Average Gini Scaling

Old scale

Used in: Old Gini Site basic equations

After opponent adjustment, old offensive and defensive Gini grades were rescaled to a 70 average.

Formula / calculation:

```

Gini multiplier_70 = 70 / mean(unadjusted Gini Stat across all 32 teams)
Adjusted Offensive Grade = Gini Stat Offense * Gini multiplier_70
Adjusted Defensive Grade = Gini Stat Defense * Gini multiplier_70

```

Legacy Simple Overall Gini

Old overall

Used in: Old Gini Site basic equations

The simple overall score combined the old adjusted offensive and defensive grades, then applied another 70-average scaling step.

Formula / calculation:

```

Gini Stat Simple Overall = Adjusted Offensive Grade + Adjusted Defensive Grade
Simple Overall multiplier_70 = 70 / mean(unadjusted overall Gini across all 32 teams)
Simple Adjusted Grade = Gini Stat Simple Overall * Simple Overall multiplier_70

```

Legacy Passing Offense Gini

[Old advanced split](#)

Used in: Old Gini Site advanced equations

The old passing offense split used passing volume, passing touchdowns, attempts, completion percentage, interceptions, opponent defensive EStat, and pass touchdowns.

Formula / calculation:

```
Pass/O base = (Games + Total Pass Yds + Pass Yds/G + Pass TDs
+ Pass Attempts + Pass Attempts/G) / The Number
Gini Stat Pass/O = (Pass/O base + 0.001*Completion_Pct - 0.001*Interceptions)
* Average Defensive EStat of Teams Played So Far
+ 0.01*Total Pass TDs
```

Note: Parentheses on the old site are loose; this is written as the clean conceptual equation.

Legacy Passing Defense Gini

[Old advanced split](#)

Used in: Old Gini Site advanced equations

The old passing defense split inverted passing production allowed and added completion suppression, interceptions, allowed yardage/TD context, and opponent offensive EStat.

Formula / calculation:

```
Pass/D allowed bundle = Games + Total Pass Yds Allowed + Pass Yds/G Allowed
+ Pass TDs Allowed + Pass Attempts Allowed + Pass Att/G Allowed
Gini Stat Pass/D = [The Number / Pass/D allowed bundle
+ 0.001*(100 - Completion_Pct_Allowed)
+ 0.001*Interceptions
+ 0.00005*Total Pass Yds Allowed
+ 0.01*Pass TDs Allowed*Average Offensive EStat Played]
* 0.5
```

Legacy Rushing Offense Gini

[Old advanced split](#)

Used in: Old Gini Site advanced equations

The old rushing offense split used rushing volume, rushing touchdowns, rush attempts, fumbles, opponent defensive EStat, and a touchdown bonus.

Formula / calculation:

```
Rush/O base = (Games + Total Rush Yds + Rush Yds/G + Rush TDs
+ Total Rush Attempts + Rush Attempts/G) / The Number
Gini Stat Rush/O = (Rush/O base - 0.001*Team Fumbles)
+ 0.05*Rush TDs*Average Defensive EStat Played*0.01
```

Legacy Rushing Defense Gini

[Old advanced split](#)

Used in: Old Gini Site advanced equations

The old rushing defense split inverted rushing production allowed and added fumble recoveries and opponent offensive strength.

Formula / calculation:

```
Rush/D allowed bundle = Games + Total Rush Yds Allowed + Rush Yds/G Allowed
+ Rush TDs Allowed + Rush Attempts Allowed + Rush Att/G Allowed
Gini Stat Rush/D = (The Number / Rush/D allowed bundle + 0.10*Fumble Recoveries)
* Average Offensive EStat Played*0.01
```

Legacy Advanced Overall Gini

Old advanced overall

Used in: Old Gini Site advanced equations

The advanced overall score combined adjusted EStats with pass/rush offensive and defensive split scores, then blended the advanced result with the simple adjusted grade.

Formula / calculation:

```
Advanced Overall raw = [((Adjusted EStat Offense*Avg Defensive EStat Played)
+ (Adjusted EStat Defense*Avg Adjusted Defensive EStat Played)*0.001)*0.005]
+ 0.060*Gini Pass/O
+ 0.055*Gini Pass/D
+ 0.045*Gini Rush/O
+ 0.040*Gini Rush/D
```

```
Advanced multiplier_70 = 70 / mean(Advanced Overall raw across all 32 teams)
Adjusted Advanced Grade Overall = Advanced Overall raw * Advanced multiplier_70
Adjusted Gini Stat Advanced Overall Grade = (Adjusted Advanced Grade Overall + Simple Adjusted Grade) / 2
```

Note: The old site notes this final metric was meant to combine all advanced metrics into one team evaluation.

Legacy Penalty Ratio

Old context input

Used in: Old EStat and key innovations

The old model treated penalty ratio as a hidden-yardage/context input rather than a standalone discipline stat.

Formula / calculation:

```
Penalty Ratio = Team Forward Penalty Yards / Team Allowed Penalty Yards
```

Note: The old site framed this as a way to contextualize whether penalties reflected aggressive style or lack of discipline.

Legacy Schedule Difficulty Integration

Old context input

Used in: Old basic and advanced Gini stats

The old model integrated schedule strength through opponent-specific offensive and defensive EStats.

Formula / calculation:

```
Offensive schedule context = Average Defensive EStat of Teams Played So Far
Defensive schedule context = Average Offensive EStat of Teams Played So Far
```

Note: This is the major conceptual bridge between the old Gini Stat and the current site's opponent-adjusted approach.

Old Site 2025-2026 Summary Stats

Old Site Top Performer

[Legacy finding](#)

Used in: Old Gini Site 2025-2026 key findings

The old site listed Detroit Lions as the top performer by Adjusted Gini Stat Advanced (overall).

Formula / calculation:

Detroit Lions Adjusted Gini Stat Advanced (Ovr.) = 93.7

Old Site Schedule Impact

[Legacy finding](#)

Used in: Old Gini Site 2025-2026 key findings

The old site reported a measurable but not overriding relationship between average schedule difficulty and overall performance.

Formula / calculation:

correlation(Average Schedule Difficulty, Overall Performance) = -0.26

Old Site Performance Balance

[Legacy finding](#)

Used in: Old Gini Site 2025-2026 key findings

The old site identified Houston Texans as having the tightest offense/defense balance.

Formula / calculation:

Houston Texans adjusted-grade gap = 0.13 points

Old Site Top 5 Overall Teams

[Legacy table](#)

Used in: Old Gini Site 2025-2026 top 5

The old site listed these five teams as the highest overall performers in the 2025-2026 view.

Formula / calculation:

1. Detroit Lions: Overall Gini 93.705, Schedule Difficulty 5.639
2. Chicago Bears: Overall Gini 92.813, Schedule Difficulty 5.523
3. Los Angeles Rams: Overall Gini 90.200, Schedule Difficulty 5.645
4. Houston Texans: Overall Gini 89.936, Schedule Difficulty 5.727
5. New England Patriots: Overall Gini 89.602, Schedule Difficulty 5.219

Old Site Divisional Schedule Difficulty

[Legacy finding](#)

Used in: Old Gini Site divisional schedule section

The old site summarized division-level schedule difficulty.

Formula / calculation:

Hardest Division = AFC South
Easiest Division = AFC East
Hardest-to-easiest division gap = 0.388

How To Read The Legacy Section

Use this old-site addendum as historical documentation of the first public Gini Stat framework. The current site has moved toward cleaner standardized EStat scores, opponent-adjusted EPA, customizable weights, live roster scoring, predictive modeling, and Super Square gates, but the old formulas show the original design philosophy: production plus opponent context plus hidden football factors.